



Phase -6



Table of Contents

1. Modern ES6+ Features (Deep Dive)
2. Modules
3. Error Handling
4. Useful Built-ins
5. Storage and Persistence
6. Best Practices

1. Modern ES6+ Features (Deep Dive)

ES6 (2015) ke baad JavaScript mein bahut saare powerful features aaye jo code ko **chhota, saaf aur readable** banate hain. Yeh features aaj har real project mein use hote hain, isliye inhe achhe se samajhna zaroori hai.

1.1 Destructuring (Advanced Patterns)

Destructuring ka matlab hai — kisi array ya object se values ko nikaal kar alag-alag variables mein daalna, ek hi line mein.

Basic recap

```
// Array destructuring
const numbers = [10, 20, 30];
const [a, b, c] = numbers;
console.log(a, b, c); // 10 20 30
```

```
// Object destructuring
const user = { name: "Aman", age: 22 };
const { name, age } = user;
console.log(name, age); // Aman 22
```

Default values

Agar value `undefined` ho, to default value use ho jaati hai.

```
const { name = "Guest", role = "user" } = { name: "Riya" };
console.log(name); // Riya (value mil gayi)
console.log(role); // user (default laga, kyunki role tha hi nahi)

// Array mein bhi
const [x = 1, y = 2, z = 3] = [10, 20];
console.log(x, y, z); // 10 20 3
```

⚠ **Important:** Default sirf tab lagta hai jab value `undefined` ho. `null`, `0`, `""` ke liye default NAHI lagega.

```
const { count = 5 } = { count: 0 };
console.log(count); // 0 (default nahi laga, kyunki 0 undefined nahi hai)
```

Renaming (naya naam dena)

```
const user = { name: "Aman", age: 22 };
const { name: userName, age: userAge } = user;
console.log(userName); // Aman
// console.log(name); // ❌ Error – ab 'name' exist hi nahi karta
```

Renaming + Default ek saath

```
const { theme: appTheme = "dark" } = {};  
console.log(appTheme); // dark
```

Nested Destructuring

Object ke andar object, ya array ke andar array — sabko ek hi line mein nikaal sakte ho.

```
const student = {  
  id: 1,  
  info: {  
    name: "Priya",  
    address: {  
      city: "Bhopal",  
      pin: 462001,  
    },  
  },  
};  
  
const {  
  info: {  
    name,  
    address: { city, pin },  
  },  
} = student;  
  
console.log(name); // Priya  
console.log(city); // Bhopal  
console.log(pin); // 462001
```

 **Note:** Upar `info` aur `address` khud variables NAHI bante — woh sirf “raasta” (path) batate hain andar tak pahunchne ka. Sirf `name`, `city`, `pin` variables bante hain.

Nested array destructuring

```
const colors = ["red", ["green", "lime"], "blue"];
const [primary, [secondary]] = colors;
console.log(primary); // red
console.log(secondary); // green
```

Function parameters mein destructuring

Yeh pattern real projects mein bahut zyada use hota hai.

```
// Pehle (purana tarika)
function greet(user) {
  console.log("Hello " + user.name + ", age " + user.age);
}

// Ab (destructuring ke saath)
function greetNew({ name, age = 18 }) {
  console.log(`Hello${name}, age${age}`);
}

greetNew({ name: "Sahil" }); // Hello Sahil, age 18
greetNew({ name: "Neha", age: 25 }); // Hello Neha, age 25
```

Swapping variables (bina temp variable ke)

```
let p = 1, q = 2;
[p, q] = [q, p];
console.log(p, q); // 2 1
```

Skipping items (kuch values chhodna)

```
const arr = [1, 2, 3, 4, 5];
const [first, , third, , fifth] = arr;
console.log(first, third, fifth); // 1 3 5
```

1.2 Spread and Rest (...)

Dono ka symbol same hai (...) lekin kaam ulta hai:

- **Spread** → cheezon ko *kholta/faila* deta hai (unpack).
- **Rest** → bachi hui cheezon ko *ikathha* karta hai (collect).

Spread — Arrays

```
const arr1 = [1, 2, 3];
const arr2 = [4, 5, 6];

// Merge / combine
const merged = [...arr1, ...arr2];
console.log(merged); // [1, 2, 3, 4, 5, 6]

// Copy (shallow copy)
const copy = [...arr1];

// Beech mein items add karna
const withZero = [0, ...arr1, 4];
console.log(withZero); // [0, 1, 2, 3, 4]
```

Spread — Objects

```
const base = { name: "Aman", age: 22 };

// Copy + extra property
const extended = { ...base, city: "Bhopal" };
console.log(extended); // { name: "Aman", age: 22, city: "Bhopal" }

// Override / update (baad waali jeet-ti hai)
const updated = { ...base, age: 23 };
console.log(updated); // { name: "Aman", age: 23 }
```

💡 **Real use case:** State update karte time (jaise React mein) — purana data spread karke sirf ek field change karna.

```
const oldState = { loggedIn: false, user: null };
const newState = { ...oldState, loggedIn: true };
```

Spread — Function arguments

```
const nums = [5, 1, 9, 3];
console.log(Math.max(...nums)); // 9
// Math.max har number alag-alag chahta hai, array nahi – spread isi liye kaam aaya
```

Spread — String ko array banana

```
console.log([... "abc"]); // ['a', 'b', 'c']
```

Rest — Function parameters

Jab pata na ho kitne arguments aayenge, tab rest sabko ek array mein daal deta hai.

```
function sum(...numbers) {
  return numbers.reduce((total, n) => total + n, 0);
}
console.log(sum(1, 2, 3)); // 6
console.log(sum(10, 20, 30, 40)); // 100
```

Rest with normal parameters

Rest hamesha **last** mein hi aata hai.

```
function intro(first, ...others) {
  console.log("Leader:", first);
  console.log("Team:", others);
}
```

```

}
intro("Aman", "Riya", "Sahil");
// Leader: Aman
// Team: ["Riya", "Sahil"]

```

Rest in destructuring

```

// Array
const [head, ...tail] = [1, 2, 3, 4];
console.log(head); // 1
console.log(tail); // [2, 3, 4]

// Object – kuch properties nikaal kar baaki ekattha karna
const { password, ...safeUser } = {
  name: "Aman",
  email: "a@x.com",
  password: "secret123",
};
console.log(safeUser); // { name: "Aman", email: "a@x.com" }
(password hata diya 👍)

```

⚠️ **Common confusion:** Spread aur Rest ko jagah se pehchaano:

- ... jab **right side / call** mein ho → **Spread** (faila raha hai)
- ... jab **left side / parameter** mein ho → **Rest** (ikattha kar raha hai)

1.3 Template Literals

Basic recap (backticks)

```

const name = "Aman";
const age = 22;
console.log(`Mera naam${name} hai aur main${age} saal ka hoon.`);

```

```
// Multi-line bhi easily
const msg = `Line 1
Line 2
Line 3`;
```

1.4 Optional Chaining (?.)

Problem: agar object ke andar ki property exist hi na ho, to `.` use karne par error aa jaata hai.

```
const user = { name: "Aman" };
// console.log(user.address.city); // ❌ TypeError: Cannot read
// properties of undefined
```

Solution: `?.` — agar pehli cheez `null` ya `undefined` ho, to aage badhega hi nahi, seedhe `undefined` return kar dega (crash nahi hoga).

```
const user = { name: "Aman" };
console.log(user.address?.city); // undefined (koi error nah
i ✅)
```

Nested ke saath

```
const data = {
  user: {
    profile: {
      social: { twitter: "@aman" },
    },
  },
};

console.log(data.user?.profile?.social?.twitter); // @aman
```



```
console.log(data.user?.settings?.theme); // undefined (crash nahi)
```

Functions ke saath (`?.()`)

Agar function exist karta hai tabhi call karo.

```
const obj = {  
  greet() { return "Hi!"; },  
};  
console.log(obj.greet?.()); // Hi!  
console.log(obj.sayBye?.()); // undefined (sayBye exist nahi karta, par crash nahi)
```

Array index ke saath (`?.[]`)

```
const arr = null;  
console.log(arr?.[0]); // undefined (crash nahi)
```

⚠ **Note:** `?.` sirf `null` aur `undefined` ko handle karta hai. Baaki cases (jaise number par property access) abhi bhi normally behave karenge.

1.5 Nullish Coalescing (`??`)

`??` ka matlab: "agar left side `null` ya `undefined` ho, tabhi right side ki value lo."

```
const name = null;  
console.log(name ?? "Guest"); // Guest  
  
const age = 0;  
console.log(age ?? 18); // 0 (kyunki 0, null/undefined nahi hai)
```

`??` vs `||` — sabse important difference 🔥

`||` falsy values (`0`, `""`, `false`, `null`, `undefined`, `NaN`) sabhi par right side leta hai.
`??` SIRF `null` aur `undefined` par right side leta hai.

```
const count = 0;

console.log(count || 10); // 10 ❌ (0 ko galti se "khaali" m
aan liya)
console.log(count ?? 10); // 0 ✅ (0 ek valid value hai)

const text = "";
console.log(text || "default"); // default
console.log(text ?? "default"); // "" (empty string valid ha
i)
```

💡 **Rule of thumb:** Jab `0`, `""`, ya `false` bhi valid values ho sakti hain, tab `??` use karo, `||` nahi.

Optional chaining + nullish coalescing ek saath (super common combo)

```
const user = { name: "Aman" };
const city = user.address?.city ?? "City not available";
console.log(city); // City not available
```

1.6 Short-Circuit Evaluation (`&&`, `||` as control flow)

JavaScript mein `&&` aur `||` sirf true/false nahi dete — woh **actual value** return karte hain, aur “short-circuit” karte hain (matlab zaroorat na ho to aage evaluate hi nahi karte).

`||` — “pehli truthy value do, warna last”

```
console.log(0 || "fallback"); // "fallback"
console.log("Aman" || "Guest"); // "Aman"
```

```
console.log(null || undefined || 5); // 5
```

Use case — default value (purana style):

```
function greet(name) {  
  name = name || "Guest"; // agar name khaali ho to Guest  
  console.log("Hello " + name);  
}
```

&& — “agar sab truthy hain to last value do, warna pehli falsy”

```
console.log(1 && 2 && 3);           // 3  
console.log(1 && 0 && 3);           // 0 (yahin ruk gaya)  
console.log("a" && "b");           // "b"
```

Use case — conditional execution (kuch tab hi karo jab condition true ho):

```
const isLoggedIn = true;  
isLoggedIn && console.log("Welcome back!"); // chalega  
  
const user = { name: "Aman" };  
user && user.name && console.log(user.name); // safe access
```

💡 React mein yeh pattern bahut dikhega:

```
{isLoggedIn && <Dashboard />} // login hai to hi dikhao
```

1.7 Logical Assignment Operators (**??=** , **||=** , **&&=**)

Yeh logical operators aur assignment ko combine karte hain — short aur clean code.

||= (OR assign)

“Agar variable falsy hai, to nayi value assign karo.”

```
let username = "";
username ||= "Guest";
console.log(username); // Guest

let score = 50;
score ||= 100;
console.log(score); // 50 (already truthy)
```

??= (Nullish assign)

“Agar variable `null` ya `undefined` hai, tabhi nayi value assign karo.”

```
let config = { theme: null, fontSize: 14 };

config.theme ??= "dark";
config.fontSize ??= 16;

console.log(config); // { theme: "dark", fontSize: 14 }
// fontSize change nahi hua kyunki 14 null/undefined nahi tha
✓
```

&&= (AND assign)

“Agar variable truthy hai, tabhi nayi value assign karo.”

```
let user = { name: "Aman", isActive: true };
user.isActive &&= false; // sirf tab badlega jab pehle se truthy ho
console.log(user.isActive); // false
```

Comparison table

Operator	Assign karta hai jab...
<code>`a</code>	

Operator	Assign karta hai jab...
<code>a ??= b</code>	<code>a</code> null ya undefined ho
<code>a &&= b</code>	<code>a</code> truthy ho

2. Modules

Jab project bada hota hai, to saara code ek file mein rakhna bilkul galat hai.

Modules ka matlab — code ko alag-alag files mein todna, aur jahan zaroorat ho wahan import kar lena. Isse code reusable, organized aur maintainable banta hai.

2.1 ES Modules — `import` / `export`

Yeh modern, official standard hai (browser aur Node dono mein chalta hai).

Setup note: Browser mein module use karne ke liye script tag mein

`type="module"` likhna zaroori hai:

```
<script type="module" src="app.js"></script>
```

Node mein `package.json` mein `"type": "module"` likho, ya file ka extension `.mjs` rakho.

Named Exports (ek file se kai cheezein export karna)

```
//  mathUtils.js
export const PI = 3.14159;

export function add(a, b) {
  return a + b;
}

export function multiply(a, b) {
  return a * b;
}
```

Importing (naam exactly same hone chahiye, curly braces mein):

```
// app.js
import { PI, add, multiply } from "./mathUtils.js";

console.log(PI);           // 3.14159
console.log(add(2, 3));    // 5
```

Alternative: ek hi jagah sab export karna

```
// mathUtils.js
const PI = 3.14159;
function add(a, b) { return a + b; }
function multiply(a, b) { return a * b; }

export { PI, add, multiply };
```

Import ko rename karna (**as**)

```
import { add as addNumbers, PI as pi } from "./mathUtils.js";
console.log(addNumbers(1, 2)); // 3
```

Sab kuch ek saath import (**as**)

```
import * as MathUtils from "./mathUtils.js";
console.log(MathUtils.PI);           // 3.14159
console.log(MathUtils.add(2, 2));    // 4
```

Default Export (ek file ka "main" export)

Har file mein sirf **ek** default export ho sakta hai.

```
// User.js
export default class User {
```

```

    constructor(name) {
      this.name = name;
    }
  }
}

```

Import karte time curly braces NAHI lagte, aur naam apni marzi ka rakh sakte ho:

```

// 📄 app.js
import User from "./User.js"; // koi bhi naam chalega
const u = new User("Aman");
console.log(u.name); // Aman

```

Default + Named ek saath

```

// 📄 api.js
export default function fetchData() { /* ... */ }
export const BASE_URL = "<https://api.example.com>";

```

```

// 📄 app.js
import fetchData, { BASE_URL } from "./api.js";

```

Re-exports (ek file se doosri file ke exports aage bhejhna)

Bade projects mein ek "index" file banayi jaati hai jo sab kuch ek jagah se export karti hai — isse imports clean rehte hain.

```

// 📄 components/index.js
export { default as Button } from "./Button.js";
export { default as Card } from "./Card.js";
export * from "./helpers.js";

```

```

// 📄 app.js – ab sab ek hi jagah se
import { Button, Card } from "./components/index.js";

```

💡 Key points yaad rakho:

- Named export → curly braces `{ }` zaroori, naam match karna chahiye.
- Default export → no braces, naam kuch bhi.
- Ek file mein 1 default + kitne bhi named exports ho sakte hain.

2.2 CommonJS (`require` / `module.exports`) — Node context

ES Modules se pehle Node.js `CommonJS` system use karta tha. Aaj bhi bahut purane Node projects aur npm packages mein yeh dikhega, isliye pehchaan-na zaroori hai.

```
// 📄 mathUtils.js (export karna)
function add(a, b) { return a + b; }
const PI = 3.14159;
```

```
module.exports = { add, PI };
```

```
// 📄 app.js (import karna)
const { add, PI } = require("./mathUtils.js");
console.log(add(2, 3)); // 5
```

Single value export:

```
// 📄 logger.js
module.exports = function (msg) {
  console.log("[LOG]:", msg);
};
```

```
const log = require("./logger.js");
log("Hello"); // [LOG]: Hello
```


ES Modules vs CommonJS — quick comparison

	ES Modules	CommonJS
Export	<code>export</code> / <code>export default</code>	<code>module.exports</code>
Import	<code>import ... from</code>	<code>require()</code>
Kahan	Browser + modern Node	Mostly older Node
Loading	Asynchronous, static	Synchronous, runtime
Status	✅ Modern standard	Legacy (still common)

💡 **Aaj ki advice:** Naya code likho to **ES Modules** use karo. CommonJS sirf tab jab purane Node project ya kisi package ki zaroorat ho.

2.3 Dynamic Imports

Normal `import` file ke top par likha jaata hai aur hamesha load ho jaata hai. Lekin kabhi-kabhi hum module ko **sirf zaroorat padne par** (on-demand) load karna chahte hain — performance ke liye. Iske liye `import()` function use hota hai.

`import()` ek **Promise** return karta hai.

```
// Button click hone par hi heavy module load karo
button.addEventListener("click", async () => {
  const module = await import("./heavyChart.js");
  module.renderChart();
});
```

`.then()` ke saath:

```
import("./mathUtils.js")
  .then((module) => {
    console.log(module.add(2, 3)); // 5
  })
  .catch((err) => console.error("Load failed:", err));
```

Conditional loading (zaroorat ke hisaab se)

```

async function loadLanguage(lang) {
  if (lang === "hi") {
    const hindi = await import("./locales/hindi.js");
    return hindi.default;
  } else {
    const english = await import("./locales/english.js");
    return english.default;
  }
}

```

💡 **Real-world benefit (Code Splitting):** Shuru mein chhota bundle load hota hai (page jaldi khulta hai), aur bada code sirf tab aata hai jab user ko zaroorat ho. Isse app fast feel hoti hai.

3. Error Handling

Real apps mein cheezein galat ho hi jaati hain — network fail, galat user input, file na mile, etc. Agar error handle na karo to pura program crash kar jaata hai. **Error handling** ka matlab — errors ko gracefully pakadna aur sambhaalna, taaki app chalti rahe.

3.1 `try` , `catch` , `finally`

```

try {
  // Code jo error de sakta hai
  const data = JSON.parse("{ invalid json }");
} catch (error) {
  // Yahan error pakda jaata hai
  console.log("Kuch galat hua:", error.message);
} finally {
  // Yeh HAMESHA chalega – chahe error aaye ya na aaye
}

```

```
    console.log("Cleanup ho gaya");
}
```

- **try** → risky code yahan dalte hain.
- **catch** → agar **try** mein error aaye to control yahan aata hai. **error** object mein details hoti hain.
- **finally** → error ho ya na ho, yeh block hamesha chalta hai (cleanup ke liye perfect — jaise file band karna, loading spinner hatana).

Practical example

```
function getUserAge(jsonString) {
  try {
    const user = JSON.parse(jsonString);
    return user.age;
  } catch (error) {
    console.log("Invalid data mila:", error.message);
    return null; // safe fallback
  }
}
```

```
console.log(getUserAge('{"age": 25}')); // 25
console.log(getUserAge("garbage"));    // Invalid data mila:
... / null
```

⚠ **Yaad rakho:** **try/catch** sirf **synchronous** errors pakadta hai. Async code ke liye **async/await** ke saath **try/catch** use karo, ya promises mein **.catch()**.

```
async function loadData() {
  try {
    const res = await fetch("<https://api.example.com/data>");
    const data = await res.json();
    return data;
  }
}
```

```

    } catch (err) {
      console.log("Fetch fail:", err.message);
    }
  }
}

```

3.2 **throw** — apni error khud bhejho

throw se hum jaan-bujhkar error pheke sakte hain, jab humein lage ki kuch galat hai.

```

function divide(a, b) {
  if (b === 0) {
    throw new Error("Zero se divide nahi kar sakte!");
  }
  return a / b;
}

try {
  console.log(divide(10, 2)); // 5
  console.log(divide(10, 0)); // yahan error throw hoga
} catch (err) {
  console.log("Error:", err.message); // Error: Zero se divid
  e nahi kar sakte!
}

```

💡 Technically kuch bhi throw kar sakte ho (string, number), par **hamesha Error object throw karo** — usme **message**, **name**, aur **stack** (kahan error aaya) ki proper info hoti hai.

```

throw "kuch galat"; // ❌ bura practice
throw new Error("kuch galat"); // ✅ achha practice

```

3.3 Built-in Error Types

JavaScript ke paas pehle se kuch error types hain. Yeh samajhne mein madad karte hain ki galti kis tarah ki hai.

Error Type	Kab aata hai	Example
Error	General/base error	<code>throw new Error("...")</code>
TypeError	Value ka type galat ho	<code>null.toUpperCase()</code>
ReferenceError	Variable exist hi na kare	<code>console.log(xyz)</code> (xyz defined nahi)
SyntaxError	Code likhne mein galti	<code>JSON.parse("{bad}")</code>
RangeError	Value allowed range ke bahar	<code>new Array(-1)</code>
URIError	URI functions galat use ho	<code>decodeURIComponent("%")</code>

```
// TypeError example
try {
  let x = null;
  x.toUpperCase();
} catch (e) {
  console.log(e.name);    // TypeError
  console.log(e.message); // Cannot read properties of nul
  l...
}

// ReferenceError example
try {
  console.log(notDefinedVariable);
} catch (e) {
  console.log(e.name); // ReferenceError
}
```

Error type ke hisaab se alag handling

```
try {
  // ... koi risky code
} catch (err) {
  if (err instanceof TypeError) {
```

```

    console.log("Type ki problem hai");
  } else if (err instanceof RangeError) {
    console.log("Value range ke bahar hai");
  } else {
    console.log("Koi aur error:", err.message);
  }
}

```

3.4 Custom Error Classes

Apne project ke liye apni khud ki error types bana sakte ho. Yeh `Error` class ko `extend` karke banti hain. Isse errors zyada meaningful aur handle-karne-mein-easy ho jaate hain.

```

class ValidationError extends Error {
  constructor(message, field) {
    super(message);          // parent Error ka constructor call
    this.name = "ValidationError"; // apna naam
    this.field = field;       // extra custom info
  }
}

class NotFoundError extends Error {
  constructor(message) {
    super(message);
    this.name = "NotFoundError";
  }
}

```

Use karna:

```

function registerUser(user) {
  if (!user.email) {
    throw new ValidationError("Email zaroori hai", "email");
  }
}

```

```

    }
    if (!user.password) {
        throw new ValidationError("Password zaroori hai", "password");
    }
    // ... registration logic
}

try {
    registerUser({ email: "a@x.com" }); // password missing
} catch (err) {
    if (err instanceof ValidationError) {
        console.log(`${err.field} field mein problem:${err.message}`);
        // password field mein problem: Password zaroori hai
    } else {
        console.log("Unknown error:", err.message);
    }
}

```

💡 **Fayda:** Custom errors se hum alag-alag tarah ki galtiyon ko alag-alag tarike se handle kar sakte hain, aur code padhne mein saaf rehta hai.

4. Useful Built-ins

JavaScript mein kuch built-in tools hain jo daily kaam mein bahut aate hain. Yahan teen important hain: `Date`, Regular Expressions, aur Iterators/Generators.

4.1 The `Date` Object

Date aur time ke saath kaam karne ke liye `Date` object use hota hai.

Date banana

```
const now = new Date(); // abhi ka time
const specific = new Date("2025-01-15"); // specific date
const detailed = new Date(2025, 0, 15, 10, 30); // year, month (0-based!), day, hour, min

console.log(now);
```

⚠ **Sabse badi trap:** Months **0 se start** hote hain! January = 0, December = 11. Days normal (1-31) hote hain.

```
new Date(2025, 0, 1); // 1 January 2025
new Date(2025, 11, 25); // 25 December 2025
```

Date se information nikaalna (getters)

```
const d = new Date("2025-06-04T10:30:00");

console.log(d.getFullYear()); // 2025
console.log(d.getMonth()); // 5 (June, kyunki 0-based)
console.log(d.getDate()); // 4 (din of month)
console.log(d.getDay()); // 3 (week ka din: 0=Sunday, 3=Wednesday)
console.log(d.getHours()); // 10
console.log(d.getMinutes()); // 30
console.log(d.getTime()); // milliseconds (1 Jan 1970 se ab tak)
```

Date set karna

```
const d = new Date();
d.setFullYear(2030);
d.setMonth(11); // December
console.log(d.getFullYear()); // 2030
```


Formatting (display ke liye)

```
const d = new Date("2025-06-04T10:30:00");

console.log(d.toLocaleDateString()); // 6/4/2025 (locale ke h
isaab se)
console.log(d.toLocaleTimeString()); // 10:30:00 AM
console.log(d.toISOString()); // 2025-06-04T10:30:00.0
00Z (standard format)

// Indian format
console.log(d.toLocaleDateString("en-IN")); // 4/6/2025
```

Practical — do dates ke beech ka difference (kitne din)

```
const start = new Date("2025-01-01");
const end = new Date("2025-01-31");

const diffMs = end - start; // milliseconds m
ein
const diffDays = diffMs / (1000 * 60 * 60 * 24); // din mein
convert
console.log(diffDays); // 30
```

💡 **Tip:** Real projects mein dates handle karna painful ho sakta hai. Production mein log `date-fns` ya `Day.js` jaisi libraries use karte hain. Lekin basics jaanna zaroori hai.

5. Storage and Persistence

Normally jab page refresh hota hai ya band hota hai, to JavaScript ke saare variables gayab ho jaate hain. **Storage** se hum data ko browser mein save kar sakte hain jo refresh ke baad bhi rahe.

5.1 localStorage aur sessionStorage

Dono ka API bilkul same hai. Sirf farak data **kab tak rehta hai** mein hai:

	localStorage	sessionStorage
Kab tak rehta hai	Hamesha (manually delete na karo tab tak)	Tab/window band hone tak
Refresh ke baad	Data rehta hai ✓	Data rehta hai ✓
Tab band karne par	Data rehta hai ✓	Data delete ho jaata hai ✗
Capacity	~5-10 MB	~5-10 MB

Methods (dono ke liye same)

```
// 1. Data save karna
localStorage.setItem("username", "Aman");

// 2. Data nikaalna
const name = localStorage.getItem("username");
console.log(name); // Aman

// 3. Ek item delete karna
localStorage.removeItem("username");

// 4. Sab kuch clear karna
localStorage.clear();
```

⚠ **Bahut zaroori:** localStorage mein sirf **strings** save hoti hain! Object ya array save karna ho to pehle `JSON.stringify()` karo, aur padhte time `JSON.parse()`.

Object/Array save karna (JSON ke saath)

```
const user = { name: "Aman", age: 22, city: "Bhopal" };

// Save karte time – object ko string banao
localStorage.setItem("user", JSON.stringify(user));
```

```
// Padhte time – string ko wapas object banao
const savedUser = JSON.parse(localStorage.getItem("user"));
console.log(savedUser.name); // Aman
console.log(savedUser.city); // Bhopal
```

sessionStorage example

```
sessionStorage.setItem("tempData", "yeh tab band hote hi gaya b");
console.log(sessionStorage.getItem("tempData"));
```

Practical — “remember theme” feature

```
// Theme save karna jab user toggle kare
function setTheme(theme) {
  localStorage.setItem("theme", theme);
  document.body.className = theme;
}

// Page load par saved theme apply karna
function loadTheme() {
  const saved = localStorage.getItem("theme") ?? "light";
  document.body.className = saved;
}
loadTheme();
```



Kab kaun sa use karein:

- `localStorage` → user preferences (theme, language), cart items, etc. jo lambe time tak chahiye.
- `sessionStorage` → temporary data jo sirf ek session ke liye ho (jaise multi-step form ka beech ka data).

- ❌ Kabhi bhi sensitive data (password, tokens) plain storage mein mat rakho — JavaScript se koi bhi padh sakta hai.

5.2 Cookies (Brief)

Cookies storage ka purana tarika hai. Yeh chhote data ke tukde hote hain jo browser save karta hai aur **har request ke saath server ko bhi bhejhe jaate hain** (yeh localStorage se sabse bada farak hai).

```
// Cookie set karna
document.cookie = "username=Aman; expires=Fri, 31 Dec 2025 2
3:59:59 GMT; path=/";

// Saari cookies padhna (ek string mein milti hain)
console.log(document.cookie); // "username=Aman; theme=dark"
```

	Cookies	localStorage
Size	~4 KB (chhota)	~5-10 MB (bada)
Server ko jaata hai	Haan (har request mein)	Nahi
Expiry set kar sakte	Haan	Nahi (manual delete)
Main use	Authentication, sessions	Client-side data

💡 **Aaj kal:** Direct cookie handling raw JavaScript se thodi tedious hai. Authentication ke liye cookies abhi bhi use hoti hain (zyadatar server set karta hai). Client-side simple data ke liye **localStorage zyada easy aur common hai**.

6. Best Practices

Code "chal jaana" kaafi nahi hai — code **saaf, samajhne layak, aur maintain karne layak** hona chahiye. Ek professional developer aur beginner ka yahi farak hota hai. Yeh practices har project mein follow karo.

6.1 Writing Clean, Readable Code

1. Functions chhote rakho — ek function, ek kaam.

```
// ❌ Bura — ek function bahut kuch kar raha hai
function processUser(user) {
  // validate, save, send email, log... sab kuch ek jagah
}

// ✅ Achha — har kaam alag function
function validateUser(user) { /* ... */ }
function saveUser(user) { /* ... */ }
function sendWelcomeEmail(user) { /* ... */ }
```

2. Magic numbers se bacho — constants use karo.

```
// ❌ Bura — 86400 kya hai? kisi ko nahi pata
if (timeDiff > 86400) { /* ... */ }

// ✅ Achha — naam se clear hai
const SECONDS_IN_A_DAY = 86400;
if (timeDiff > SECONDS_IN_A_DAY) { /* ... */ }
```

3. Nesting kam karo — early return use karo.

```
// ❌ Bura — gehra nesting (pyramid of doom)
function getDiscount(user) {
  if (user) {
    if (user.isPremium) {
      if (user.years > 2) {
        return 50;
      }
    }
  }
  return 0;
}

// ✅ Achha — early return, flat code
```

```
function getDiscount(user) {
  if (!user) return 0;
  if (!user.isPremium) return 0;
  if (user.years > 2) return 50;
  return 0;
}
```

4. Comments “kyun” batayein, “kya” nahi. Code khud bata dega ki kya ho raha hai. Comment tab likho jab koi cheez non-obvious ho.

```
// ❌ Faltu comment
let count = 0; // count ko 0 set kiya

// ✅ Useful comment
// API rate limit avoid karne ke liye 1 sec wait
await delay(1000);
```

6.2 Naming Conventions

Achhe naam aadhi documentation hote hain. Naam se hi pata chal jana chahiye ki cheez kya hai.

Cheez	Convention	Example
Variables / functions	camelCase	userName, getTotal()
Constants (fixed)	UPPER_SNAKE_CASE	MAX_SIZE, API_URL
Classes	PascalCase	UserAccount, HttpClient
Boolean	is/has/can se shuru	isActive, hasPermission
Private (convention)	_ se shuru	_internalCount

```
// ✅ Achhe naam – padhte hi samajh aata hai
const isLoggedIn = true;
const maxRetryCount = 3;
function calculateTotalPrice(items) { /* ... */ }
```

```
// ❌ Bure naam – kuch samajh nahi aata
const x = true;
const n = 3;
function calc(a) { /* ... */ }
```

💡 **Golden rule:** Naam itna clear ho ki comment ki zaroorat hi na pade. `d` likhne ki jagah `daysRemaining` likho.

6.3 Avoiding Common Pitfalls

1. `var` ki jagah `const` / `let` use karo. `var` ke scoping issues hote hain. Default `const`, change karna ho to `let`. `var` bhool jao.
2. `==` ki jagah `===` use karo. `==` type conversion karta hai aur weird results deta hai.

```
console.log(0 == ""); // true 😞 (== ka chakkar)
console.log(0 === ""); // false ✅ (=== safe hai)
console.log(1 == "1"); // true 😞
console.log(1 === "1"); // false ✅
```

3. Loop ke andar `async/await` se bacho jab parallel chal sakta ho.

```
// ❌ Slow – ek-ek karke (serial)
for (const url of urls) {
  await fetch(url);
}

// ✅ Fast – sab ek saath (parallel)
await Promise.all(urls.map(url => fetch(url)));
```

4. Object/Array reference se share hota hai — galti se modify mat karo.

```
const original = { count: 1 };
const copy = original;
copy.count = 99;
```

```
console.log(original.count); // 99 🤖 (dono same object hai n!)
```

```
// ✅ Proper copy  
const safeCopy = { ...original };
```

5. Floating point se bacho exact comparison mein.

```
console.log(0.1 + 0.2); // 0.30000000000000004 🤖  
console.log(0.1 + 0.2 === 0.3); // false!  
// Solution: toFixed() ya thoda tolerance use karo  
console.log((0.1 + 0.2).toFixed(2)); // "0.30"
```

6.4 Debugging with DevTools

Browser ke DevTools (F12 ya Right-click → Inspect) developer ka sabse bada dost hai. `console.log` se aage badho aur yeh seekho:

Breakpoints (sabse powerful tool):

- DevTools → **Sources** tab kholo.
- Code ki line number par click karo → wahan ek breakpoint lag jaata hai.
- Ab jab code us line par pahunchega, woh **ruk** jaayega. Tum us waqt har variable ki value dekh sakte ho.

Step controls (jab code ruka ho):

- **Step over (F10)** → agli line par jao (function ke andar nahi ghuso).
- **Step into (F11)** → agar line par function hai, to uske andar ghuso.
- **Step out** → current function se bahar niklo.
- **Resume (F8)** → agle breakpoint tak normal chalo.

Watch:

- "Watch" panel mein koi bhi expression add kar do (jaise `user.age * 2`).

- Code chalte time uski value live update hoti rahegi. Baar-baar `console.log` likhne se bach jaate ho.

Call Stack inspection:

- Call Stack panel batata hai ki **abhi tum yahan kaise pahunche** — kaunsa function kisko call kiya.
- Sabse upar current function, neeche jisne use call kiya, aur aise hi neeche tak.
- Bug dhoondhne mein bahut help karta hai — “yeh function call kahan se hua?”

`console` ke aur tarike:

```
console.log("normal");
console.error("error (laal dikhega)");
console.warn("warning (peela)");
console.table([{ name: "A", age: 1 }, { name: "B", age: 2 }]); // table format!
console.group("Group"); // related logs ko group karna
console.groupEnd();
```

💡 **Pro tip:** `console.log` se debugging chalti hai, lekin **breakpoints zyada powerful** hain — woh exact moment par sab kuch freeze karke dikha dete hain. Thodi practice karke seekho, time bahut bachega.

6.5 Reading and Understanding Error Messages

Beginners error message dekhte hi ghabra jaate hain. Lekin error message tumhara **dost** hai — woh exactly bata raha hai kahan aur kya galat hai. Padhna seekho:

```
Uncaught TypeError: Cannot read properties of undefined (reading 'name')
    at getUser (app.js:15:23)
    at main (app.js:42:5)
```

Ise tod kar samjho:

1. `TypeError` → error ka type (yahan: kisi cheez ka type galat tha).

2. `Cannot read properties of undefined (reading 'name')` → asli problem: tum `undefined` cheez par `.name` padhne ki koshish kar rahe ho. Matlab koi object expect kiya tha, par `undefined` mil gaya.
3. `at getUser (app.js:15:23)` → ye **line 15, column 23** par `getUser` function mein hua. Yahan dekho!
4. **Call stack** → `main` ne `getUser` ko call kiya tha. Raasta pata chal gaya.

Aam errors aur unka matlab:

Error	Matlab	Aam wajah
<code>Cannot read properties of undefined</code>	undefined par <code>.something</code> access kiya	object exist nahi karta / typo
<code>x is not defined</code>	variable declare hi nahi hai	spelling galat / scope ka issue
<code>x is not a function</code>	jo function samjha woh function nahi	typo / galat import
<code>Unexpected token</code>	syntax galat	missing bracket, comma, etc.
<code>Maximum call stack exceeded</code>	infinite recursion	function khud ko endlessly call kar raha

💡 Steps jab error aaye:

1. Error ka **type** padho.
2. Error ki **line aur file** dekho (sabse pehli `at` line jo tumhari file ki ho).
3. Us line par jao aur dekho kya `undefined` /galat ho sakta hai.
4. Phir bhi samajh na aaye to error message **Google karo** — 99% chance hai kisi aur ne yeh face kiya hoga (Stack Overflow).